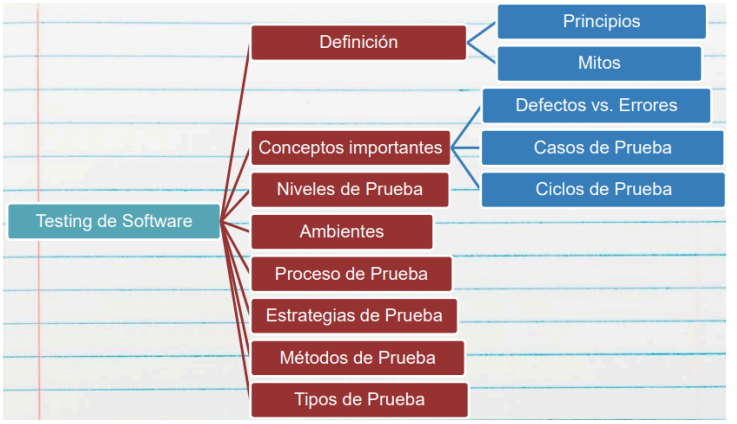
TESTING DE SOFTWARE

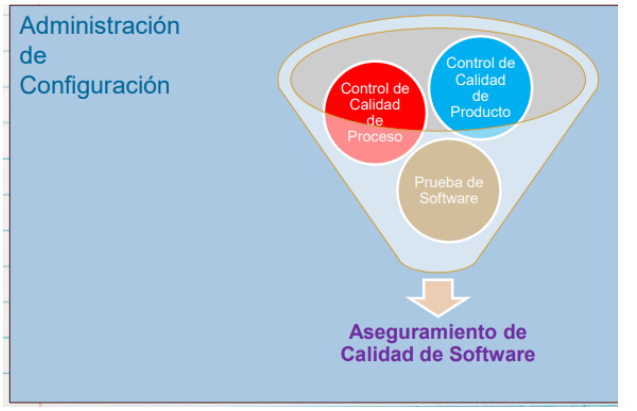
Cobertura de temas dentro del contexto de testing:



En las clases prácticas se trabaja con Estrategias de Prueba.

PRUEBA DE SOFTWARE EN CONTEXTO…

La prueba (testing) de software se ubica dentro del contexto del Aseguramiento de Calidad de Software (SQA), pero es una disciplina distinta del Control de Calidad de Proceso y del Control de Calidad de Producto.



Es decir, no se reduce a ninguna de las dos: constituye un tema en sí mismo. Asegurar la calidad no equivale a "probar el software al final". Probar es una herramienta más entre varias.

Aun así, se desarrolla bajo el paraguas de la Administración (Gestión) de Configuración de Software, que da el marco para saber qué se controla y qué se prueba.

¿QUÉ ES EL TESTING?

El **testing es un proceso cuyo objetivo es encontrar defectos en el software**. Su propósito **no es** demostrar que el software funciona correctamente ni asegurar que no existen defectos.

Por eso parte de una premisa: **Se asume que existen defectos en el código.**

Por eso se lo considera un **proceso destructivo**: se intenta poner al software en situaciones que permitan **hacer aparecer sus defectos**.

¿Qué actitud se debe tener al hacer Testing?

Se debe adoptar una **actitud negativa**, es decir, intentar demostrar que **algo está mal o es incorrecto**, en lugar de buscar confirmar que todo funciona.

El tester debe preguntarse, por ejemplo: ¿Cómo puedo hacer que esto falle? ¿Qué pasa si ingreso un dato inválido? ¿Qué pasa en una situación límite? ¿Qué condición podría romper esta funcionalidad?

¿Cuándo es exitoso un Testing?

Un **testing es exitoso cuando encuentra defectos**. Es decir, que un test encuentre errores es algo positivo.

Si el test encuentra un defecto, cumplió justamente con su objetivo: **hacer visible un problema que estaba presente en el software**.

Testing ≠ asegurar calidad

El testing **no garantiza que el software esté libre de defectos**. Que no se hayan encontrado defectos durante las pruebas **no significa que no existan**.

El testing NO puede asegurar ni la calidad del software ni que se obtenga un software de calidad.
La calidad se construye **durante todo el proceso de desarrollo**, no al final mediante testing.

Relación entre desarrollo y testing

Un desarrollo no exitoso puede conducir a un testing exitoso.

Porque si durante el desarrollo se introdujeron defectos, el testing puede encontrarlos. Entonces, **el éxito del testing se mide por su capacidad de descubrir problemas**, no por que el software "pase" todas las pruebas.

Desarrollo con defectos → Testing encuentra defectos → Testing exitoso

Mientras que: **Testing sin encontrar defectos ≠ Software sin defectos**

Costo
El testing puede representar aproximadamente **entre el 30% y el 50%** del costo de desarrollar un software confiable.

¿Qué es el testing? - IEEE / ISTQB

El **testing** es un proceso que comprende actividades realizadas **a lo largo de todo el ciclo de vida del software**, tanto **estáticas como dinámicas**:

- Testing estático: analiza los productos de trabajo **sin ejecutar el software**. Por ejemplo, revisar requerimientos, diseño o código para detectar problemas.
- Testing dinámico: implica **ejecutar el software** bajo determinadas condiciones, observar los resultados y evaluarlos para detectar defectos.

Esto es importante porque **testing no significa solamente ejecutar el programa y ver si funciona**.

Podemos resumir las definiciones de ISTQB e IEEE en tres ideas:

1. comprobar si se cumplen los **requerimientos especificados**,
2. evaluar si el software es **apto para su propósito**,
3. **detectar defectos**.

ERROR VS DEFECTO

No son sinónimos. La diferencia principal está en **cuándo se introduce el problema y cuándo se detecta.**

Error → es un problema introducido durante una determinada etapa del desarrollo y **detectado dentro de esa misma etapa**.
Ejemplo: Estoy implementando una funcionalidad y, al hacer una revisión de código, encuentro que escribí mal una condición. Ese problema es un error, porque fue introducido y detectado en la misma etapa.

Defecto → es un error que **no fue detectado en la etapa en la que se introdujo** y se traslada a una etapa posterior.
Ejemplo: Durante la implementación se introduce un problema en el código, pero no se detecta mediante las revisiones. Más adelante, durante el testing, se descubre. Ese problema ahora se considera un defecto.

Un **defecto puede generar una falla**, pero **no necesariamente todo defecto genera una falla**. Esto puede ocurrir porque el defecto está en una parte del código que nunca se ejecuta o porque no se dan las condiciones necesarias para que se manifieste.

Por eso, en el contexto del testing, hablamos principalmente de **detectar defectos**, porque estamos encontrando problemas que **sobrevivieron a etapas anteriores**.

Clasificar el defecto

Cuando durante el testing encontramos un **defecto**, debemos clasificarlo para decidir **cómo y cuándo abordarlo**.
Se tienen en cuenta dos aspectos:

1. Severidad: Indica **qué tan grave es el defecto para el funcionamiento del sistema** y cuál es el impacto que tiene.

Clasificación:
Bloqueante → impide continuar con el uso del sistema o con una actividad fundamental. (el sistema no anda)
Crítico → tiene un impacto muy grave sobre una funcionalidad importante. (el sistema anda)
Mayor → afecta de manera importante el funcionamiento de una funcionalidad, pero no llega a ser crítico. (hay una alternativa, o plan B)
Menor → tiene un impacto reducido y no afecta significativamente el funcionamiento. (seguir operando igual solo que con una falla presente)
Cosmético → afecta principalmente aspectos visuales o de presentación.

2. Prioridad: Indica **qué tan urgente es resolver el defecto**.
   No depende directamente de la severidad. La prioridad se determina según el **contexto**, teniendo en cuenta las necesidades del **negocio, usuario o cliente** y qué tan rápido se necesita la solución.

   Clasificación:
   Urgencia
   Alta
   Media
   Baja

   Severidad ≠ Prioridad

   Un defecto **muy severo no necesariamente tiene que ser el de mayor prioridad**, y un defecto de baja severidad puede tener una prioridad alta dependiendo del contexto.

   Por ejemplo: el sistema se cae al realizar una operación fundamental.
   → Severidad: Bloqueante
   → Si esa funcionalidad recién se va a utilizar dentro de un mes, podría no ser la prioridad inmediata.

NIVELES DE PRUEBA

Los **niveles de prueba** permiten definir **qué parte del software estamos probando y con qué alcance**.

La idea es que, al hacer testing, vamos **de lo más pequeño a lo más grande**, integrando progresivamente los componentes:

Componente → Integración → Sistema → Aceptación

Es decir, a medida que subimos de nivel, **aumenta el alcance de lo que estamos probando**.

Primer nivel - Pruebas unitarias

Se prueba un **componente individual**, de manera acotada y, en lo posible, aislada del resto del sistema. Por ejemplo: Tengo una función que calcula el precio final de una entrada → pruebo específicamente esa función.

- Generalmente las realizan los **desarrolladores**.
- Son de las pruebas **más fáciles de automatizar**.
- Se realizan durante la **implementación**. El desarrollador construye y lo prueba.
- Permiten detectar problemas muy cerca del momento en que fueron introducidos, por ende, se encuentran **errores** más que defectos.

Se prueba cada componente tras su realización/construcción. Los errores se suelen reparar tan pronto como se encuentran, sin constancia oficial de los incidentes.

Segundo nivel - Pruebas de integración

Test orientado a verificar que las partes de un sistema que funcionan bien aisladamente, también lo hacen en conjunto. Es decir, se prueban **componentes integrados**, verificando que funcionen correctamente **al comunicarse entre sí**. La integración es de componentes que ya pasaron por las pruebas unitarias.

Generalmente se realizan **de manera incremental**, integrando los componentes progresivamente. Puede ser:

- Integración de arriba hacia abajo (top-down) → arranca desde el módulo principal (interfaz o lógica superior)
- Integración de abajo hacia arriba (bottom-up) → arranca desde lo más simple (BD, funciones puras)

Lo ideal es una combinación de ambos esquemas.

El resultado de integrar los componentes que queremos probar es un **Build**, es decir, una versión del software que contiene un conjunto de componentes integrados.

- Generalmente lo ejecutan los **testers.**
- Los **módulos críticos** deben ser probados lo más **tempranamente** posible.
- Se debe **minimizar la necesidad de programas auxiliares** utilizados para simular componentes que todavía no están integrados.

Tercer nivel - Pruebas de sistema

Prueba de la construcción Final. Es un nivel de prueba de **mayor alcance**, donde se prueba el sistema como un todo. Se busca verificar que el sistema, considerado globalmente, **funcione como se espera**.

Se trabaja sobre el conjunto de componentes que ya fueron integrados y que resuelven un determinado conjunto de requerimientos. Es decir, sobre el Build.

- Generalmente las realizan **testers**.
- Se prueba el sistema en un entorno lo más parecido posible al **entorno de producción** para reducir al mínimo el riesgo de incidentes debidos al ambiente. Es decir, asegurar que el ambiente o la infraestructura no rompa el programa.

Cuarto nivel - Pruebas de aceptación

Acá hay una diferencia importante con los otros niveles: **el objetivo no está centrado únicamente en encontrar defectos**, sino en determinar si el sistema es aceptable y generar confianza en que satisface las necesidades del usuario.

A diferencia de los niveles anteriores, estas pruebas suelen ser realizadas por el **usuario final o cliente**.

Se realizan en un entorno de **preproducción/despliegue**, antes de que el sistema sea puesto en producción.

Comprende tanto la prueba realizada por el usuario en ambiente de laboratorio *(pruebas alfa)*, como la prueba en ambientes de trabajo reales *(pruebas beta).*

AMBIENTES PARA CONSTRUCCIÓN DEL SOFTWARE

Anteriormente, en el tercer nivel, hablamos de que el sistema se prueba en un entorno lo más parecido posible al entorno de producción. Aca veremos bien, cuales son los ambientes para la construcción del software.

Un **ambiente o entorno** es el espacio/condiciones de trabajo que se prepara para poder realizar una determinada actividad sobre el software. Es decir, los lugares donde se trabaja para construir software.

Un ambiente no es solamente una computadora o un servidor. Es el conjunto de hardware, sistema operativo, herramientas, librerías, versiones, configuraciones y demás recursos necesarios para realizar una determinada actividad sobre el software.

Cada actividad puede requerir un entorno diferente:

- Desarrollo → Donde los desarrolladores construyen el software, **principalmente escribiendo y modificando código** (implementar funcionalidades, corregir errores, realizar pruebas unitarias).
- Prueba → Es un entorno destinado a **probar el software**. Por ejemplo, pueden realizarse allí **pruebas de integración**. Algunas pruebas unitarias también pueden ejecutarse en este ambiente, aunque muchas se realizan directamente en desarrollo. El ambiente de prueba intenta ser **similar al de producción**, pero no necesariamente tiene exactamente las mismas características (como requisitos de performance,concurrencia,redundancia,etc.).
- Pre-Producción → Normalmente se realizan aquí las **pruebas de sistema** y, dependiendo del caso, también pueden realizarse **pruebas de aceptación**. **Es el último entorno de validación antes de la producción.** Es un ambiente destinado a realizar pruebas sobre una versión del software muy cercana a la que posteriormente se utilizará en producción.
- Producción → Es el ambiente donde el software **opera realmente y es utilizado por sus usuarios**. Es el entorno que representa el funcionamiento real del sistema. En determinados casos, pueden realizarse pruebas de aceptación en producción, pero esto puede ocurrir únicamente cuando el software todavía no está siendo utilizado efectivamente por el mercado/usuarios.

No es una correspondencia rígida: una prueba no necesariamente se ejecuta exclusivamente en un único ambiente. Por ejemplo, las unitarias pueden ejecutarse en desarrollo o prueba.

CASOS DE PRUEBA

Un caso de prueba es un conjunto de instrucciones que describen paso a paso un escenario de uso de una funcionalidad del sistema para determinar si funciona correctamente, comparando el **resultado obtenido con el resultado esperado.**

En la práctica, un caso de prueba debe **especificar concretamente qué acciones realizar y con qué valores**, para que la prueba pueda ejecutarse de manera sistemática y reproducible.

Ejemplo: caso de prueba para iniciar sesión

- Acción: seleccionar la opción «Iniciar sesión».
- Usuario: `angiambroggio`.
- Contraseña: `1234`.
- Resultado esperado: Si el usuario no existe o la contraseña es incorrecta, el sistema debería rechazar el inicio de sesión y mostrar el mensaje correspondiente. Si existe, debería dejar iniciar sesión.

No alcanza con escribir «ingresar un usuario y una contraseña». Hay que especificar los valores concretos que se van a utilizar.

Los casos de prueba permiten:

- **Determinar si el software funciona correctamente**, comparando los resultados obtenidos con los esperados.
- **Descubrir defectos** que no se detectaron durante las etapas anteriores.
- **Reproducir defectos**, registrando los pasos, las condiciones y los datos necesarios para volver a provocar el problema.
- Ejecutar las pruebas de forma **sistemática**, en lugar de probar sin un procedimiento definido.

Si el caso de prueba está bien definido es más fácil *reproducir defectos.* La reproducibilidad es fundamental: si encontramos un defecto, necesitamos poder repetir los pasos que lo provocaron para que el equipo pueda analizarlo y corregirlo, lo cual no se logrará sin un buen caso de prueba definido.
Si un defecto no se puede reproducir, resulta mucho más difícil identificar su causa y corregirlo.

Definir los Casos de Prueba

El desafío o inconveniente es que un sistema puede tener muchísimas combinaciones de datos, acciones y condiciones. Probar absolutamente todas las posibilidades se conoce como **testing exhaustivo**, y en sistemas reales suele ser **inviable por el tiempo y el costo que requiere.** Es decir, no puedo definir casos de prueba infinitos, tengo que tener algún

mecanismo para lograr definir la menor cantidad de casos de prueba que me permitan cubrir el testing de manera completa o lo más completa posible.

Por eso, el objetivo es seleccionar una cantidad razonable de casos que permita **maximizar la detección de defectos con el menor esfuerzo y tiempo posibles,** cubriendo los aspectos relevantes del software. Para lograrlo, se utilizan distintas **estrategias de prueba** (que vemos en el práctico), que permiten seleccionar los casos de manera sistemática.

OBJETIVO	**descubrir errores**
CRITERIO	**en forma completa**
RESTRICCIÓN	**con el mínimo de esfuerzo y tiempo**

Más allá de las estrategias de testing, una idea clásica asociada a Boris Beizer es arrancar la selección de los casos basándose en que:
Los defectos suelen esconderse en las esquinas y concentrarse en los límites. Entonces conviene prestar especial atención a los valores límite o bordes de los casos de prueba.

Por ejemplo: una compra admite importes desde $100 hasta $100.000.

Podríamos probar:

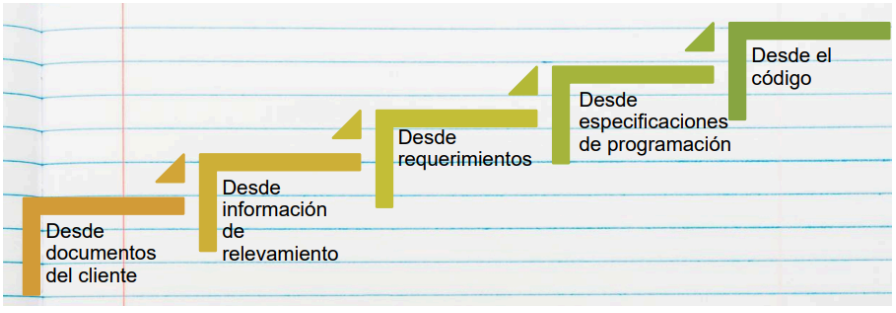
- Un valor justo por debajo del mínimo: $99,99.
- El mínimo permitido: $100.
- Un valor válido intermedio: $50.000.
- El máximo permitido: $100.000.
- Un valor justo por encima del máximo: $100.000,01.

Así comprobamos si el sistema acepta y rechaza correctamente los valores correspondientes.

Esta es una **idea inicial / premisa** que sirve para diseñar pruebas; más adelante, las estrategias de testing permiten aplicarla de forma sistemática.

Derivación de casos de prueba

Los casos de prueba se pueden derivar de distintas fuentes, según la información disponible sobre el software:



Cuanto más completa y precisa sea la documentación, más fácil será diseñar los casos de prueba.

Por ejemplo, si los requerimientos están bien detallados, podemos derivar los casos de prueba directamente a partir de ellos. En cambio, si son generales o poco precisos, tendremos que definir con mayor detalle en los propios casos de prueba aspectos del comportamiento esperado que no quedaron especificados previamente.

En otras palabras, **la falta de requerimientos claros dificulta el testing**, porque no tenemos una referencia precisa contra la cual comparar el comportamiento del sistema.

Conclusiones sobre la generación de casos de prueba

La idea es que los casos de prueba no se definan arbitrariamente, sino siguiendo un criterio y dejando constancia de las decisiones tomadas.

Ninguna técnica de prueba es completa: cada técnica permite detectar determinados tipos de defectos, pero puede no encontrar otros.

Las técnicas atacan distintos problemas: por eso, lo recomendable es combinar varias técnicas para complementar sus ventajas.

La técnica depende del código o sistema que se va a probar: no todas son igualmente adecuadas para cualquier situación.

Sin requerimientos, el testing es mucho más difícil: cuesta determinar qué comportamiento debería tener el sistema.

Tener en cuenta la conjetura de defectos: consiste en utilizar la experiencia y el conocimiento sobre errores frecuentes para anticipar dónde podrían encontrarse defectos.

Ser sistemático y documentar las suposiciones: hay que registrar cómo interpretamos el comportamiento esperado y qué posibles fallas consideramos, especialmente cuando la información disponible es incompleta.

Condiciones de prueba

Una condición de prueba es una característica, situación o aspecto del sistema que debe comprobarse.
Está relacionada con una determinada respuesta esperada del sistema frente a un estímulo, este estímulo está constituido por las distintas **entradas.**

Primero identificamos **qué queremos probar** y después diseñamos uno o más casos de prueba para comprobarlo.

Por ejemplo, en un inicio de sesión podemos identificar estas condiciones de prueba:

- El sistema debe permitir el acceso cuando las credenciales son correctas.
- El sistema debe rechazar el acceso cuando la contraseña es incorrecta.
- El sistema debe rechazar el acceso cuando el usuario no existe.

Una vez definidas, en función de estas defino el caso de prueba.

Condición de prueba → Caso(s) de prueba

- Condición: qué comportamiento o aspecto quiero comprobar.
- Caso de prueba: cómo lo voy a comprobar, con qué datos y qué resultado espero obtener.

Una condición de prueba debe estar cubierta por al menos un caso de prueba. Una misma condición puede necesitar varios casos para verificar distintas situaciones.

Estrategias/Métodos de los Casos de Prueba

Como dijimos, **el tiempo y el presupuesto disponibles para el testing son limitados.** Como no podemos probar absolutamente todas las posibilidades, necesitamos seleccionar los casos de prueba de forma sistemática.

El objetivo es **cubrir la mayor cantidad posible de funcionalidades y detectar la mayor cantidad posible de defectos con la menor cantidad de pruebas y esfuerzo.**

Por eso, existen dos grandes enfoques **complementarios** que buscan alcanzar ese objetivo:

Caja negra:

Se basa en las especificaciones (entrada) y en el comportamiento esperado del software (salida). Permiten diseñar casos de prueba sin analizar la estructura interna del código.

Se llama «caja negra» porque no nos interesa cómo está implementada internamente la funcionalidad: nos concentramos en **qué recibe el sistema** y **qué debería hacer.**

Se divide en 2 técnicas:

- *Basado en especificaciones*: Se utilizan los requerimientos o las especificaciones para decidir qué casos de prueba realizar.
  - Partición de Equivalencias
  - Análisis de valores límites
  - Etc.
- *Basados en la experiencia:* Se apoyan en la experiencia y el conocimiento del tester para anticipar dónde pueden aparecer defectos. Estas técnicas se nombran como introducción, pero no son el foco de la práctica que van a desarrollar, no lo vemos en la práctica tampoco.
  - Adivinanza de Defectos
  - Testing Exploratorio

Caja blanca:

Se basan en analizar la **estructura interna del software o de un componente**, es decir, cómo está construido el código.

A diferencia de la caja negra, acá sí interesa conocer la implementación para diseñar pruebas que recorran determinadas partes del código y verifiquen su comportamiento.

Técnicas:

- Cobertura de enunciados o caminos básicos.
- Cobertura de sentencias.
- Cobertura de decisión.
- Cobertura de condición.
- Cobertura de decisión/condición.
- Cobertura múltiple.

Estas técnicas buscan medir qué partes de la estructura interna se ejercitaron durante las pruebas. El alcance de la cobertura que se puede garantizar depende de la técnica y de los criterios aplicados: **tener cobertura no garantiza que se hayan encontrado todos los defectos.**

Caja negra	Caja blanca
Se basa en especificaciones y comportamiento esperado.	Se basa en la estructura interna y el código.
No necesita conocer la implementación interna.	Requiere conocer la implementación.
Se centra en entradas, salidas y comportamiento.	Se centra en qué partes del código se ejecutan y qué decisiones se toman.
Ejemplo: comprobar que una contraseña incorrecta sea rechazada.	Ejemplo: comprobar que se ejecuten las distintas ramas de la condición que valida la contraseña.

Concepto de Ciclo de prueba/test

Un ciclo de pruebas comprende la **ejecución de la totalidad de los casos de prueba** establecidos **sobre una misma versión del sistema.**

Por ejemplo, supongamos que tenemos 20 casos de prueba para una aplicación:

1. Se ejecutan los 20 casos sobre la versión 1.0.
2. Se registran los resultados y los defectos encontrados.
3. Se corrigen los defectos y se genera una nueva versión.
4. Se ejecutan nuevamente los casos de prueba sobre esa nueva versión.

El proceso puede repetirse hasta **alcanzar el criterio de finalización o aceptación** que se haya definido. Por ejemplo, se podría establecer que el testing finaliza cuando no quedan defectos críticos pendientes o cuando se alcanza el número de ciclos de pruebas que se definió hacer. Esto es necesario porque **no se puede probar indefinidamente hasta garantizar que no existen defectos.**

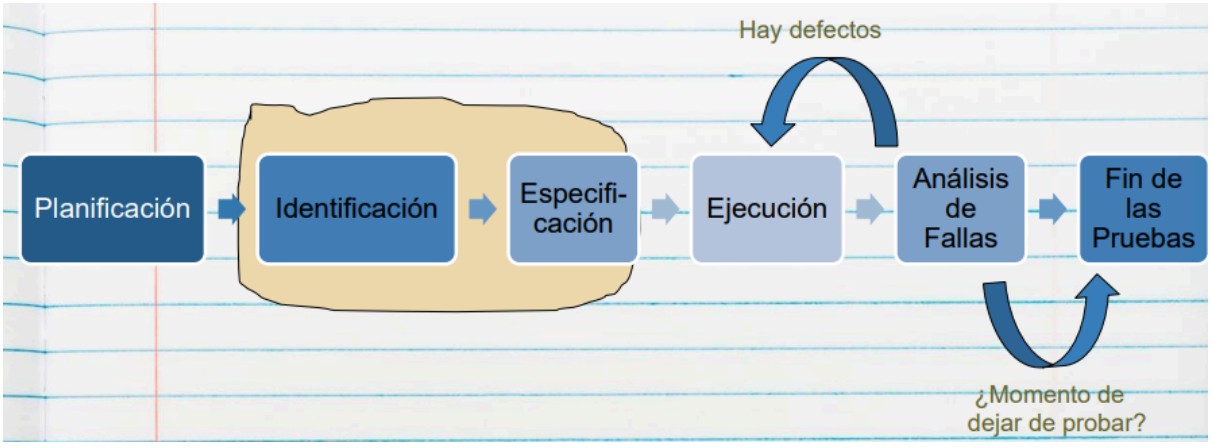
Concepto de Regresión - Pruebas de regresión

Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, debe realizarse una **verificación total** (probar nuevamente TODOS los casos de prueba) de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados.

Es decir, **corregir un defecto o modificar el código de una versión, para pasar a otra nueva versión, puede introducir nuevos defectos o afectar otras funcionalidades.** Por eso, cuando se genera una nueva versión, no alcanza necesariamente con volver a ejecutar únicamente los casos de prueba que habían fallado. También hay que considerar los casos que habían pasado correctamente.

Anotación: Siempre en las nuevas versiones si o si aplico este concepto? No, se va jugando con esto. Es decir, no necesariamente siempre se ejecuta una regresión completa. Como las pruebas consumen tiempo y recursos, el alcance de la regresión puede variar según los cambios realizados, el riesgo y el presupuesto disponible.

PROCESO DE PRUEBAS



Importante en el contexto de la parte teórica del testing.
Hablamos en un contexto de proceso definido, aunque si no está en términos de proceso definido, el concepto de como hacer el testing siempre funciona más o menos de esta manera.

El proceso de pruebas es una secuencia de actividades que permite **planificar, diseñar, ejecutar y evaluar las pruebas de un software** para detectar defectos y verificar que cumpla con los resultados esperados.

Se puede aplicar tanto en un **proceso definido o tradicional** como en un contexto ágil. **Lo que cambia es cómo se organizan y documentan las actividades, no la necesidad de probar el software.**

ETAPAS:

1. PLANIFICACIÓN Y CONTROL: parte estratégica.

Planificación: Consiste en verificar que se comprendan las metas y los objetivos del cliente, de los stakeholders (partes interesadas) y del proyecto, así como los riesgos que se pretende abordar mediante las pruebas.

Se construye el **Test Plan (plan de pruebas)**, que establece:

- Riesgos y objetivos del testing.
- Estrategia de testing: cómo se realizarán las pruebas.
- Recursos necesarios: personas, herramientas, tiempo y presupuesto.
- Criterios de aceptación: condiciones que deben cumplirse para aceptar los resultados.

Es decir, se define qué se va a probar, cómo se hará, qué recursos se necesitan y cuáles son los riesgos, los tiempos y el presupuesto disponibles. También se establecen los criterios que determinarán cuándo finalizarán las pruebas.

Control: consiste en revisar los resultados del testing y compararlos con lo planificado para determinar si se está avanzando adecuadamente. Para ello, se analiza:

- El test coverage (cobertura de pruebas): cuánto de lo que se pretende probar fue cubierto.
- El cumplimiento de los criterios de aceptación.
- Los resultados obtenidos, para tomar decisiones sobre cómo continuar.

Diferencia clave: planificar es definir qué se hará y cómo; controlar es revisar lo que está sucediendo y decidir si es necesario tomar medidas.

En Agile si se planifica pero no como un plan rígido al comienzo, no existe ese documento estático, sino que la planificación es **continua, iterativa y compartida** por todo el equipo.

2. IDENTIFICACIÓN Y ESPECIFICACIÓN: diseño

Se determinan qué se va a probar y cómo se comprobará que funciona correctamente. Esta parte también se conoce como **diseño de pruebas.**

Las actividades incluyen:

- **Revisión de la base de pruebas:** analizar los documentos y elementos utilizados como referencia para diseñar las pruebas, por ejemplo, requisitos y especificaciones.
- **Verificación de las especificaciones:** revisar qué debe hacer el software que se va a probar.
- **Evaluación de la testeabilidad:** determinar si los requisitos y el sistema permiten realizar pruebas adecuadas.
- **Identificación de los datos necesarios:** determinar qué datos de entrada se necesitan para probar los distintos escenarios.
- **Diseño y priorización de los casos de prueba:** definir los pasos, los datos de entrada y los resultados esperados, estableciendo qué casos son más importantes.

- **Diseño del entorno de prueba:** determinar cómo debe configurarse el ambiente donde se ejecutarán las pruebas.

Hasta ahora, nada de código, se preparan las pruebas, pero todavía no se ejecutan.

3. EJECUCIÓN:

Se ejecutan los casos de prueba sobre el software implementado y se comparan los resultados obtenidos con los esperados. La ejecución puede ser manual o automatizada.

Las actividades incluyen:

- Desarrollar y priorizar los casos de prueba.
- Crear los datos de prueba.
- Automatizar las pruebas cuando sea necesario.
- Agrupar los casos en conjuntos de pruebas (test suites) para ejecutarlos de manera eficiente.
- Implementar y verificar el entorno de prueba.
- Ejecutar los casos de prueba.
- Registrar los resultados e identificar el software probado, incluyendo su identidad y versión, mediante las herramientas de testing.
- Comparar los resultados reales con los resultados esperados.

Salida: Si se detectan diferencias, se registran para su análisis y corrección.

Importante: aunque la PPT ubica algunas tareas de preparación dentro de Ejecución, conceptualmente están relacionadas con el diseño y la preparación de las pruebas. En el proceso real, las actividades pueden solaparse.

4. ANÁLISIS DE FALLAS

Se investigan los resultados incorrectos para determinar qué defectos pueden estar provocándolos.

- Analizar las diferencias entre los resultados reales y los esperados.
- Registrar los defectos con información suficiente para reproducirlos.
- Facilitar su corrección.
- Volver a ejecutar las pruebas correspondientes después de corregirlos.

Cuando se corrige un defecto, **se vuelven a ejecutar** las pruebas correspondientes para comprobar que se haya solucionado. Por lo tanto, se genera un ciclo de ejecución → análisis de fallas → corrección → nueva ejecución, que se repite hasta cumplir los criterios de salida (que determinan el fin de las pruebas).

5. EVALUACIÓN, REPORTE Y FIN DE LAS PRUEBAS

Se finaliza cuando se cumplen los criterios de salida definidos previamente en la planificación. **No significa necesariamente que no quede ningún defecto.** Probar exhaustivamente todo el software suele ser inviable por cuestiones de tiempo y

presupuesto. Por ejemplo, se puede establecer como fin que no haya defectos bloqueantes ni críticos pendientes y que se haya alcanzado un nivel de cobertura determinado.

Las actividades incluyen:

- **Evaluar los criterios de aceptación:** comprobar si se cumplieron las condiciones establecidas para aceptar los resultados.
- **Reportar los resultados:** comunicar a los stakeholders el estado y los resultados de las pruebas.
- **Consolidar la información:** reunir los datos de las actividades de testing completadas.
- **Verificar los entregables y las correcciones:** comprobar que se hayan entregado los elementos previstos y que los defectos que debían corregirse hayan sido solucionados.
- **Evaluar el proceso de testing:** analizar cómo se realizaron las actividades e identificar lecciones aprendidas para mejorar futuras pruebas.

VERIFICACION Y VALIDACION

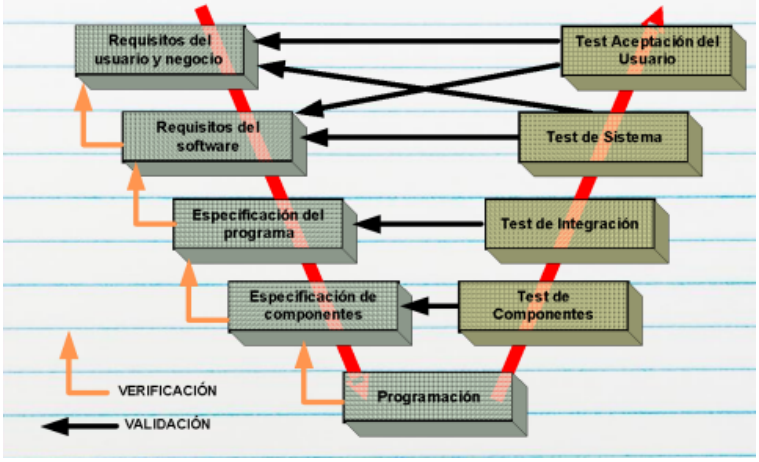
Son dos conceptos relacionados con el testing que permiten evaluar distintos aspectos del software, aca solo los vamos a nombrar, después los veremos nuevamente.

Verificación: ¿Estamos construyendo el sistema correctamente?

Consiste en evaluar si el software se está desarrollando de acuerdo con las especificaciones establecidas, evaluar si estamos construyendo el sistema libre de defectos

Validación: ¿Estamos construyendo el sistema correcto?

Consiste en comparar lo que construimos con lo que nuestro cliente quiere, por eso decimos el sistema correcto, porque el sistema se corresponde con los requerimientos, y los requerimientos se corresponden con lo que el cliente quiere.



EL TESTING Y EL CICLO DE VIDA

Las actividades de testing deben involucrarse **lo más temprano posible en el ciclo de vida del software,** incluso desde la etapa de requisitos, antes de comenzar a programar. ¿En qué momento podría empezar a trabajar con las actividades del testing? Desde el principio. Si yo tengo definido los requisitos ya puedo empezar a trabajar, es decir, no necesito ni 1 línea de código para empezar a trabajar en el testing.

Los principales objetivos de involucrar el testing tempranamente son:

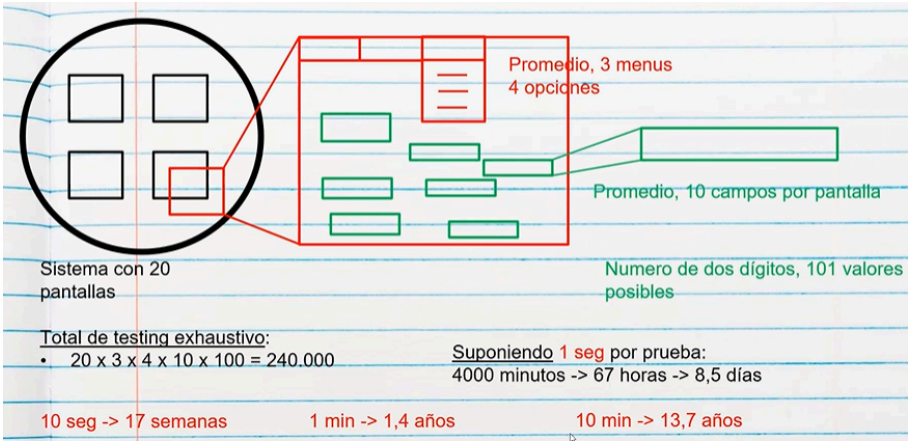
- **Dar visibilidad temprana al equipo:** permite conocer desde el principio cómo se va a probar el producto y qué aspectos se deberán verificar.
- **Disminuir los costos de corrección de defectos:** cuanto antes se detecta un defecto, generalmente más sencillo y económico resulta corregirlo, porque todavía no se propagó a otras etapas o componentes del sistema. Es decir, me puede servir para darme cuenta si hay cosas que he omitido en los casos de prueba, antes de avanzar con la implementación.

MITOS DEL TESTING

- El Testing es una etapa que comienza al terminar de codificar. Aclaramos arriba que no es así.
- El Testing es probar que el software funciona. No, el testing es encontrar defectos, no alcanza con comprobar que el software funciona, hay que buscar situaciones en las que pueda fallar.
- TESTING = CALIDAD de PRODUCTO. Ayuda a evaluar la calidad pero no puede garantizarla.
- TESTING = CALIDAD de PROCESO. No me asegura la calidad del proceso. Ayuda a que el software sea confiable al detectar sus defectos, pero no me asegura que el proceso de desarrollo haya seguido buenas prácticas durante su construcción.
- El tester es el enemigo del programador. No, tienen objetivos contrapuestos (porque, mientras el programador busca construir una solución que funcione, el tester intenta encontrar situaciones en las que esa solución no funcione como debería), pero si el tester encuentra un defecto, **no significa que esté atacando al programador ni que haya hecho mal su trabajo.** Simplemente está haciendo visible un problema que debe corregirse para mejorar la confiabilidad del producto.

¿CUÁNTO TESTING ES SUFICIENTE?

Volvemos a repetir que es imposible probar el software de manera completa, pero con un ejemplo:



El ejemplo trata de mostrar por qué **el testing exhaustivo no es viable:**
Un sistema con 20 pantallas, 10 campos por pantalla, 3 menús con 4 opciones, un sistema super pequeño y sin embargo probar TODAS las posibilidades, nos da un tiempo gigante.

Decidir cuánto tiempo es suficiente → Depende principalmente de tres factores:

- **Riesgos:** qué tan graves serían las consecuencias si el sistema falla. Por ejemplo, no es lo mismo hacer testing sobre un sistema que va a dosificar medicamentos en un paciente (un defecto podría poner en riesgo la vida de una persona), a hacer testing sobre una app de citas.
- **Costos y recursos:** cuánto tiempo y presupuesto se pueden dedicar al testing.
- **Criterios de aceptación:** qué condiciones deben cumplirse para considerar suficientes las pruebas. Por ejemplo: “Hago testing hasta que el porcentaje de test corrido sin falla sea de tanto %”, o “dedicamos este costo al testing, cuando consumimos las horas, el testing se acaba”.

Usamos los riesgos para determinar:
• Que testear primero
• A qué dedicarle más esfuerzo de testing
• Que no testear (por ahora)

PRINCIPIOS DEL TESTING

Son los que ya hemos nombrado.
Los principios del testing son ideas fundamentales que orientan cómo realizar las pruebas y qué podemos esperar de ellas.

- El testing muestra presencia de defecto.
- El testing exhaustivo es imposible.
- Testing temprano.
- Agrupamiento de Defectos. Los defectos no suelen distribuirse uniformemente por todo el software: tienden a concentrarse en determinados módulos o componentes.
- El testing es dependiente del contexto.
- Falacia de la ausencia de errores. La presencia de defectos en el software es algo que se asume, se asume que existen.
- Paradoja del Pesticida: (nuevo)

  Si ejecutamos repetidamente los mismos casos de prueba, llega un momento en que dejan de detectar defectos nuevos.

  Esto puede ocurrir incluso cuando las pruebas están automatizadas: se ejecutan siempre de la misma manera y comprueban los mismos escenarios, por lo que otros defectos pueden permanecer ocultos.

  Por eso, **es necesario revisar, actualizar y agregar casos de prueba,** incorporando nuevos escenarios y datos.

- Un programador debería evitar probar su propio código. A excepción de las pruebas unitarias, es el único caso de excepción. Una persona que desarrolló el código puede pasar por alto defectos porque conoce cómo lo construyó y tiende a probarlo según lo que espera que haga.

- Una unidad de programación no debería probar sus propios desarrollos. Refuerza la idea anterior.
- Examinar el software para probar que no hace lo que se supone que debería hacer es la mitad de la batalla, la otra mitad es ver que hace lo que no se supone que debería hacer. Actitud negativa → tiene que ver con la actitud de buscar defectos: el tester no debe limitarse a comprobar los casos exitosos.
- No planificar el esfuerzo de testing sobre la suposición de que no se van a encontrar defectos. No sería realista planificar suponiendo que los programadores hicieron todo correctamente y que ninguna prueba fallará. Esto no implica criticar el trabajo del programador: se parte de que todo software puede contener defectos y que el testing existe para ayudar a detectarlos.

SMOKE TEST

El Smoke Test es una primera ejecución rápida de pruebas sobre el sistema para comprobar que las funcionalidades principales funcionan y que el software no presenta fallas catastróficas que impidan continuar con pruebas más detalladas.

Se suele realizar cuando se recibe una nueva versión del software, antes de invertir tiempo en pruebas más exhaustivas.

Es una corrida rápida por todo el software, porque no se prueban escenarios alternativos ni casos de prueba rebuscados, sino que se prueba en escenarios generales para ver que no haya ninguna falla catastrófica. Es superficial. Es un ejemplo de tipos de prueba.

No es lo mismo que el Sanity Test. El Sanity Test suele ser más acotado y se utiliza para verificar que un cambio o una corrección específica funcione como se espera.

TIPOS DE PRUEBA

Hay dos grandes categorías:

Testing funcional

Consiste en comprobar **qué hace el sistema**, es decir, si las funcionalidades se comportan de acuerdo con los requerimientos y las necesidades definidas.
Es el tipo de prueba más “fácil”, porque si los requerimientos están planteados correctamente, se derivan directamente de ellos.
Ejemplo: Si el requisito dice que una transferencia debe descontar el dinero de una cuenta y acreditarlo en otra, el testing funcional verifica que ese comportamiento ocurra correctamente.

Testing no funcional

Consiste en comprobar **cómo funciona el sistema,** es decir, evaluar características de calidad que no se limitan a las funcionalidades que ofrece.

Los requisitos no funcionales son tan importantes como los funcionales. Un sistema puede hacer correctamente lo que debe hacer, pero resultar demasiado lento, inestable o difícil de utilizar.

Tipos:
• Performance Testing
• Pruebas de Carga
• Pruebas de Stress
• Pruebas de usabilidad
• Pruebas de mantenimiento
• Pruebas de fiabilidad
• Pruebas de portabilidad